

Define your population

Contents

- [Before starting](#)
- [Get metadata from a BIDS hierarchy with `clinica iotools`](#)
- [Prepare metadata with `clinicadl tsvtools`](#)
- [Split the data samples into training, validation and test sets](#)

```
# Uncomment the next lines if running in Google Colab
!pip install clinicadl==1.6.1
```

This notebook is an introduction to the tools proposed by ClinicaDL to identify relevant samples and to split them into coherent groups to be used during the training, the validation and the test stages.

! Important

This step is mandatory preliminary to training to avoid issues such as lack of clinical meaning or data leakage.

In the following, we will see how to split these samples between training, validation and test sets using tools available in `clinica` and `clinicadl`.

Before starting

This notebook allows to prepare the dataset to train a neural network.

These first two commands are the only ones that require access to the BIDS. If you were not able to process the data as indicated in the previous notebook, you can uncomment the following cell to download the BIDS of 4 subjects from OASIS-1 or the BIDS of 2 subjects from ADNI that were generated in the [preprocessing section](#).

```
# #OASIS BIDS
!curl -k
https://aramislab.paris.inria.fr/clinicadl/files/handbook_2023/data_oas
is/BIDS_example.tar.gz -o BIDS_example.tar.gz
!tar xf BIDS_example.tar.gz
```

```
# #ADNI BIDS
!curl -k
https://aramislab.paris.inria.fr/clinicadl/files/handbook_2023/data_adn
i/BIDS_example.tar.gz -o BIDS_example.tar.gz
!tar xf BIDS_example.tar.gz
```

Get metadata from a BIDS hierarchy with **clinica iotools**

Gather BIDS and CAPS data into a single TSV file

In a BIDS hierarchy, demographic, clinical and imaging metadata are stored in TSV files located at different levels of the hierarchy depending on whether they are specific to a subject (e.g. gender), a session (e.g. diagnosis) or a scan (e.g. acquisitions parameters).

The following command line can be used to merge all the metadata in a single TSV file:

```
clinica iotools merge-tsv <bids_directory> <output_tsv>
```

where:

- **bids_directory** is the input folder containing the dataset in a BIDS hierarchy.
- **output_tsv** is the path of the output tsv. If a directory is specified instead of a file name, the default name for the file created will be **merge-tsv.tsv**.

We are going to run some experiments on the ADNI and OASIS datasets, if you have already downloaded the full datasets and converted them to BIDS, you can set the path to the dataset directory by changing the following paths. If not, just run it as written. Execute the following command to gather metadata included in this BIDS.

```
# Merge meta-data information
!clinica iotools merge-tsv data_oasis/BIDS_example
data_oasis/merged.tsv
```

```
!clinica iotools merge-tsv data_adni/BIDS_example data_adni/merged.tsv
```

Check missing modalities for each subject

We want to restrict the list of sessions used to only include those with a T1-MR image. The following command allows to identify which modalities are available for each session:

```
clinica iotools check-missing-modalities <bids_directory>
<output_directory>
```

where:

- **bids_directory** is the input folder of a BIDS compliant dataset.
- **output_directory** is the output folder.

This pipeline does not have an option to give a list of subject/session, so it checks the missing modalities for all the datasets.

Execute the following command to find which sessions include a T1-MR image on the example BIDS of OASIS:

```
# Find missing modalities
!clinica iotools check-missing-modalities data_oasis/BIDS_example
data_oasis/missing_mods
```

```
!clinica iotools check-missing-modalities data_adni/BIDS_example
data_adni/missing_mods
```

The output of this command, `missing_mods/`, is a folder with a series of files (one file per session label containing one row per subject and one column per modality).

Prepare metadata with `clinicadl tsvtools`

In this section we will work on a subset of 100 sessions of the OASIS dataset (and a subset of 100 sessions of the ADNI dataset) and you only need the list of the sessions, for now.

The whole preprocessing process has been run for you on these datasets. The results of the [quality check procedure](#) have been used to filter sessions.

`data_oasis/oasis_after_qc.tsv` and `data_adni/adni_after_qc.tsv` store the list of the sessions that have been accepted for each dataset.

You can run the next cell to download the necessary resources (`merged.tsv` and `oasis_after_qc.tsv` - or `adni_after_qc.tsv` - files, as well as the `missing_mods` directory).

```
#for OASIS-1 dataset
!curl -k
https://aramislab.paris.inria.fr/clinicadl/files/handbook_2023/data_oasis/iotools_output.tar.gz -o iotools_output.tar.gz
!tar xf iotools_output.tar.gz
!curl https://raw.githubusercontent.com/aramislab/clinicadl_handbook/main/data/oasis_after_qc.tsv --output
data_oasis/oasis_after_qc.tsv
```

```
#for the ADNI dataset
!curl -k
https://aramislab.paris.inria.fr/clinicadl/files/handbook_2023/data_adni/iotools_output.tar.gz -o iotools_output.tar.gz
!tar xf iotools_output.tar.gz
!curl https://raw.githubusercontent.com/aramislab/clinicadl_handbook/main/data/adni_after_qc.tsv --output
data_adni/adni_after_qc.tsv
```

Get the labels

The 3 labels described in the [first part of the course](#) (AD, CN, MCI) can be extracted with ClinicaDL using the command:

```
clinicadl tsvtools get-labels <bids_directory> <results_tsv>
```

where:

- `bids_directory` the input folder containing the dataset in a BIDS hierarchy.
- `results_tsv` is the path to the tsv file.

💡 Tip

You can increase the verbosity of the command by adding `-v` flag(s).

The `bids_directory` argument is mandatory to run the `clinica iotools merge-tsv` and `clinica iotools check-missing-modalities` within this pipeline if it has not been done before. If you already have run these pipelines, the path is no longer mandatory, and you can put anything, just add the options `--merged_tsv` and `--missing_mods`, to avoid re-running these pipelines.

By default the pipeline only extracts the AD and CN labels, which corresponds to the only available labels in OASIS. Run the following cell to extract them in a new file `labels.tsv` from the restricted version of OASIS:

```
!clinicadl tsvtools get-labels data_oasis/BIDS_example data_oasis --merged_tsv data_oasis/merged.tsv --missing_mods data_oasis/missing_mods --restriction_tsv data_oasis/oasis_after_qc.tsv
```

In the ADNI dataset, a subject can have several sessions during his follow-up and so you can find another diagnosis, mild cognitive impairment (MCI). For more information please refer to the [preprocessing section](#). Moreover, the BIDS example that you have downloaded doesn't label alzheimer's disease as 'AD' but as 'Dementia' so you need to add the `--diagnosis/-d` option.

```
!clinicadl tsvtools get-labels data_adni/BIDS_example data_adni --merged_tsv data_adni/merged.tsv --missing_mods data_adni/missing_mods --restriction_tsv data_adni/adni_after_qc.tsv -d CN -d Dementia -d MCI
```

This tool writes a unique TSV file containing the labels asked by the user. They are stored in the column named diagnosis.

Restriction path:

At the end of the command line, a restriction was given to extract the labels only from sessions in `oasis_after_qc.tsv`. This tsv file corresponds to the output of the [quality check procedure](#) that was manually cut to only keep the sessions passing the quality check. It depends on the preprocessing: here it concerns a run of `t1-linear`.

Analyze the population

You may have noticed that there is a bias on age in the OASIS dataset: the youngest AD participant is 62 years old, whereas many CN participants are much younger. To correct this bias, we will exclude the youngest CN participants (i.e. who are less than 62 years old) from our data, thanks to the following Python script:

```

from os import PathLike

def remove_youngest_cn(table_path: PathLike, minimum_age: int):
    """Remove youngest CN patients to correct age bias"""
    import pandas as pd

    table = pd.read_csv(table_path, sep='\t')
    new_table = table[(table['diagnosis'] == 'AD') | (table['age_bl']
    >= minimum_age)]

    print(
        f"These participants were excluded from the dataset
    ({len(table) - len(new_table)} participants): \n\n",
        table.loc[table.index.difference(new_table.index)]
    )

    new_table.to_csv(table_path, sep='\t')

```

```
remove_youngest_cn('data_oasis/labels.tsv', minimum_age=62)
```

However, other biases may exist, especially after the quality check of the preprocessing which removed sessions from the dataset. Thus, it is crucial to check before going further if there are other biases in the dataset.

ClinicaDL implements a tool to perform a demographic and clinical analysis of the population:

```
clinicadl tsvtools analysis <merged_tsv> <data_tsv> <results_path>
```

where:

- **merged_tsv** is the output file of the **clinica iotools merge-tsv** command.
- **data_tsv** is the output file of **clinicadl tsvtool getlabels|split|kfold**.
- **results_path** is the path to the tsv file that will be written (filename included).

The following command will extract statistical values on the populations for each diagnostic label. Based on those, it is possible to check that the dataset is suitable for the classification task.

```

# Run the analysis on OASIS
!clinicadl tsvtools analysis data_oasis/merged.tsv
data_oasis/labels.tsv data_oasis/analysis.tsv

```

```

# Run the analysis on ADNI
!clinicadl tsvtools analysis data_adni/merged.tsv data_adni/labels.tsv
data_adni/analysis.tsv -d CN -d Dementia -d MCI

```

```
def display_table(table_path):
    """Custom function to display the clinicadl tsvtool analysis
    output"""
    import pandas as pd

    OASIS_analysis_df = pd.read_csv(table_path, sep='\t')
    OASIS_analysis_df.set_index("group", drop=True, inplace=True)
    columns = [
        "n_subjects",
        "n_scans",
        "mean_age",
        "std_age",
        "min_age",
        "max_age",
        "sexF",
        "sexM",
        "mean_MMSE",
        "std_MMSE",
        "min_MMSE",
        "max_MMSE",
        "CDR_0",
        "CDR_0.5",
        "CDR_1",
        "CDR_2",
        "CDR_3",
    ]
    # Print formatted table
    format_columns = ["subjects", "scans", "age", "sex", "MMSE", "CDR"]
    format_df = pd.DataFrame(index=OASIS_analysis_df.index,
                             columns=format_columns)
    for idx in OASIS_analysis_df.index.values:
        row_str = "%i; %i; %.1f ± %.1f [%i, %i]; %iF / %iM; %.1f ± %.1f [%i, %i]; 0: %i, 0.5: %i, 1: %i, 2: %i, 3: %i" %
        tuple([OASIS_analysis_df.loc[idx, col] for col in columns])
        row_list = row_str.split(';')
        format_df.loc[idx] = row_list

    format_df.index.name = None
    display(format_df)
```

```
display_table("data_oasis/analysis.tsv")
```

```
display_table("data_adni/analysis.tsv")
```

Note

If you were not able to run the previous cell to get the analysis, you can find the results in the [data](#) folder on GitHub to have an overview of what it should look like.

There is no significant bias on age anymore, but do you notice any other problems?

Demographic bias:

There is still a difference in the sex distribution and the network could learn a bias on sex such as "women are cognitively normal" and "men are demented". However, there are too few images in OASIS to continue removing sessions to balance the groups.

To check that such bias is not learnt, it is possible to run a logistic regression after training between sex and the predicted label to check if they are correlated.

Get the progression of the Alzheimer's disease

For the ADNI dataset, because the dataset is longitudinal, the stability of the diagnostic status can be computed. The progression label corresponds to the following description:

- s (stable): diagnosis remains identical during the time_horizon period following the current visit,
- p (progressive): diagnosis progresses to the following state during the time_horizon period following the current visit (eg. MCI → AD),
- r (regressive): diagnosis regresses to the previous state during the time_horizon period following the current visit (eg. MCI → CN),
- uk (unknown): there are not enough sessions to assess the reliability of the label but no changes were spotted,
- us (unstable): otherwise (multiple conversions / regressions).

ClinicaDL implements a tool to get the progression label for each couple [subject, session] and adds a new column progression to the TSV file given as argument.

```
clinicadl tsvtools get-progression [OPTIONS] DATA_TSV
```

with :

- **DATA_TSV** (str) is the TSV file containing the data (output of `clinicadl tsvtools get-labels|split|kfold`).
- **--time_horizon** (int) can be added: It is the time horizon in months that is used to assess the stability of the MCI subjects. Default value: 36.

Tip

The diagnosis column do not need to be part of the columns, the pipeline will go back to the labels.tsv to calculate the progression

Run the pipeline on the ADNI dataset

```
!clinicadl tsvtools get-progression data_adni/labels.tsv --time_horizon 36
```

```
import pandas as pd
df_labels = pd.read_csv("data_adni/labels.tsv", sep="\t")
df_labels.set_index(["participant_id", "session_id"])
print(df_labels)
```

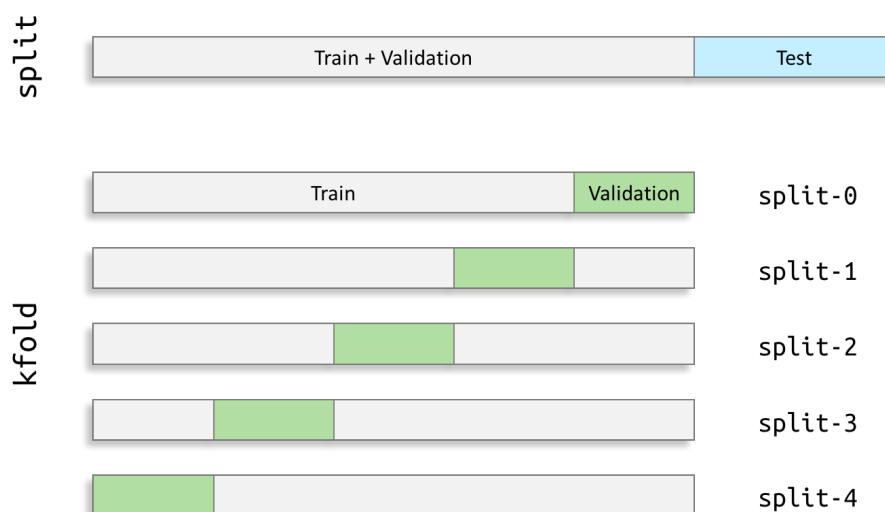
Split the data samples into training, validation and test sets

Now that the labels have been extracted and possible biases have been identified, the data has to be split in different sets. This step is essential to guarantee the independence of the final evaluation.

Definition of sets:

In this notebook, data samples are divided between train, validation and test sets:

- The **train set** is used to update the weights,
- The **validation set** is used to stop the training process and select the best model,
- The **test set** is used once the training process is finished, and is used to perform an unbiased evaluation of the performance of the model.



In the k-fold validation procedure, k trainings are conducted according to the k training/validation pairs generated. This leads to k different models that are evaluated on the test set once the training is finished. The final test performance is then the mean value of these k models.

Tools that have been developed for this part are based on the guidelines of ([Varoquaux et al., 2017](#)).

Build the test set

The test set is obtained by performing a single split obtained with `clinicadl tsvtool split`:

```
clinicadl tsvtool split <data_tsv>
```

where:

• `Print to PDF` the TSV file with the data that are going to be split (output of `clinicadl tsvtools get-labels|split|kfold`).

Each diagnosis label is split independently. Random splits are generated until the differences between age and sex distributions between the test set and the train + validation set are non-significant. Then three TSV files are written:

- the baseline sessions of the test set,
- the baseline sessions of the train + validation set,
- the longitudinal sessions of the train + validation set.

In OASIS there is no longitudinal follow-up, hence the last two TSV files are identical.

Let's create a test set including 20% of the subjects:

```
!clinicadl tsvtools split data_oasis/labels.tsv --n_test 0.2 --subset_name test
```

```
# for Adni dataset
!clinicadl tsvtools split data_adni/labels.tsv --n_test 0.2 --subset_name test
```

The differences between the populations of the train + validation and test sets can be assessed to check that there are no discrepancies between the two sets.

```
!clinicadl tsvtools analysis data_oasis/merged.tsv
data_oasis/split/train.tsv data_oasis/analysis_trainval.tsv
```

```
!clinicadl tsvtools analysis data_oasis/merged.tsv
data_oasis/split/test_baseline.tsv data_oasis/analysis_test.tsv
```

```
print("Train + validation set")
display_table("data_oasis/analysis_trainval.tsv")
print("Test set")
display_table("data_oasis/analysis_test.tsv")
```

If you are not satisfied with these populations, you can relaunch the `clinicadl tsvtools split` command and change the parameters used to evaluate the difference between the distributions: `p_age_threshold` and `p_sex_threshold`.

Unique test set:

Only one test set was created in ([Wen et al., 2020](#)) to evaluate the final performance of one model. This is because architecture search was performed on the training + validation sets. As this operation is very costly and/or is done mostly manually, it was not possible to do it several times.

Build the validation sets

To better estimate the performance of the network, it is trained 5 times using a 5-fold cross-validation procedure. In this procedure, each sample is used once to validate and the other times to train the network. In the same way as for the single split, the TSV files can be processed by ClinicaDL:

```
clinicadl tsvtool kfold <formatted_data_path>
```

where `formatted_data_path` is the output tsv file of `clinicadl tsvtool get-labels|split|kfold`.

In a similar way as for the test split, three tsv files are written **per split** for each set:

- the baseline sessions of the validation set,
- the baseline sessions of the train set,
- the longitudinal sessions of the train set.

Contrary to the test split, there is no attempt to control the similarity between the age and sex distributions. Indeed here we consider that averaging across the results of the 5 folds already reduces bias compared to a single data split.

```
!clinicadl tsvtools kfold data_oasis/split/train.tsv --n_splits 5 --subset_name validation
```

```
# for ADNI dataset
!clinicadl tsvtools kfold data_adni/split/train.tsv --n_splits 5 --subset_name validation
```

Check the absence of data leakage

In OASIS-1 there is no risk of data leakage due to the data split itself as there is only one session per subject. Also, there is no MCI patients, hence there is no risk of data leakage during a transfer learning between a source task involving the MCI set and a target task involving at least one of its subsets (sMCI or pMCI). However, for other datasets, it might be useful to check that there is no correlated data spread between the train and test sets.

A script in `clinicadl` has been created to check that there was no data leakage after the split steps. More specifically it checks that:

1. Baseline datasets contain only one scan per subject.
2. No intersection exists between train and test sets.
3. MCI train subjects are absent from test sets of subcategories of MCI.

As it is not a common function, it has not been integrated to the general command line. The next cell executes it on the splits generated in the previous sections.

```
import os
from pathlib import Path
import pandas as pd
from clinicadl.utils.tsvtools_utils import extract_baseline
"""
Check the absence of data leakage
1) Baseline datasets contain only one scan per subject
2) No intersection between train and test sets
"""

def check_is_subject_unique(labels_path_baseline: Path):
    flag_is_unique = True
    check_df = pd.read_csv(labels_path_baseline, sep="\t")
    check_df.set_index(["participant_id", "session_id"], inplace=True)
    if labels_path_baseline.name[-12:] != "baseline.tsv":
        check_df = extract_baseline(check_df, set_index=False)
    for _, subject_df in check_df.groupby(level=0):
        if len(subject_df) > 1:
            flag_is_unique = False
    if flag_is_unique:
        print(f"subject uniqueness is TRUE in {labels_path_baseline}")
    else:
        print(f"subject uniqueness is FALSE in {labels_path_baseline}")

def check_is_independent(train_path_baseline: Path, test_path_baseline: Path):
    flag_is_independent = True
    train_df = pd.read_csv(train_path_baseline, sep="\t")
    train_df.set_index(["participant_id", "session_id"], inplace=True)
    test_df = pd.read_csv(test_path_baseline, sep="\t")
    test_df.set_index(["participant_id", "session_id"], inplace=True)
```

```

    for subject, session in train_df.index:
        if (subject, session) in test_df.index:
            flag_is_independent = False
        if flag_is_independent:
            print(f"{train_path_baseline} and {test_path_baseline} are
independant.")
        else:
            print(f"{train_path_baseline} and {test_path_baseline} are NOT
independant.")

def run_test_suite(data_tsv: Path, n_splits: int):
    _run_test_suite_no_split(data_tsv) if n_splits == 0 else
_run_test_suite_multiple_splits(data_tsv)

def _run_test_suite_no_split(data_tsv: Path):
    check_train = True
    train_baseline_tsv = data_tsv / "train_baseline.tsv"
    test_baseline_tsv = data_tsv / "test_baseline.tsv"
    if not train_baseline_tsv.exists():
        check_train = False
    check_is_subject_unique(test_baseline_tsv)
    if check_train:
        check_is_subject_unique(train_baseline_tsv)
        check_is_independent(train_baseline_tsv, test_baseline_tsv)

def _run_test_suite_multiple_splits(data_tsv: Path):
    for folder, _, files in os.walk(data_tsv):
        folder = Path(folder)
        for file in files:
            if file[-3:] == ".tsv":
                check_is_subject_unique(folder / file)
                train_baseline_tsv = folder / "train_baseline.tsv"
                test_baseline_tsv = folder / "validation_baseline.tsv"
                if train_baseline_tsv.exists():
                    if test_baseline_tsv.exists():
                        check_is_independent(train_baseline_tsv,
test_baseline_tsv)

# Run check for train+val / test split
run_test_suite(Path("../data_oasis/split"), n_splits=0)

# Run check for train / validation splits
run_test_suite(Path("../data_oasis/split/5_fold"), n_splits=5)

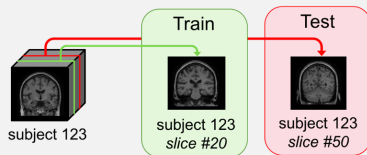
```

If no Error was raised, then none of the three conditions was broken. It is now possible to use the train and the validation sets to perform a classification task, and then to evaluate correctly the performance of the classifier on the test set.

Data leakage:

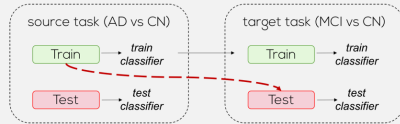
Many procedures can cause data leakage and thus bias the performance, leading to impossible claims. It is crucial to check that the test set has not been contaminated by data that is correlated to the train and/or validation sets. You will find below examples of procedures that can lead to data leakage.

Biased split



Samples (slices, patches, sessions) from the same subject are both in the train and the test sets.

Biased transfer learning



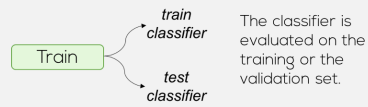
The source task and the target task share samples from the same class, and a new split was performed for the target task.

Late split



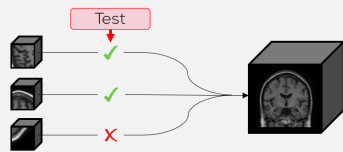
Another procedure (data augmentation, features selection, autoencoder pretraining...) was performed prior to the data split.

No independent test set



The classifier is evaluated on the training or the validation set.

Biased ensemble learning



Parts of the image are selected / weighted thanks to the label of the test set to deduce the image-level prediction of the test set.

Now that you have your train, test and validation splits, you can train a network for classification, regression or reconstruction with ClinicaDL.